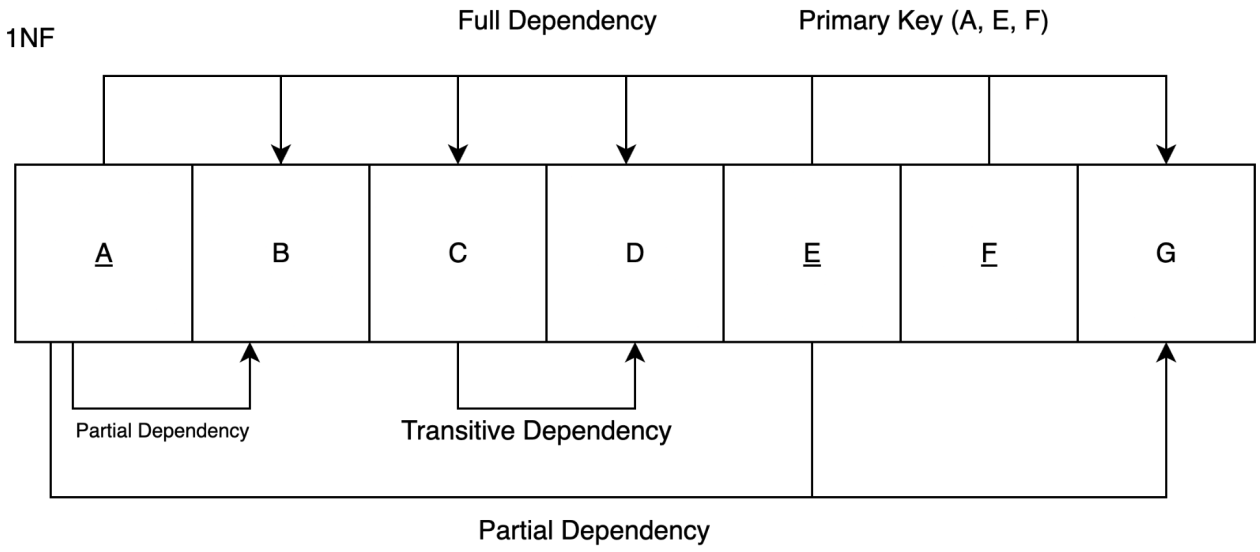
1.

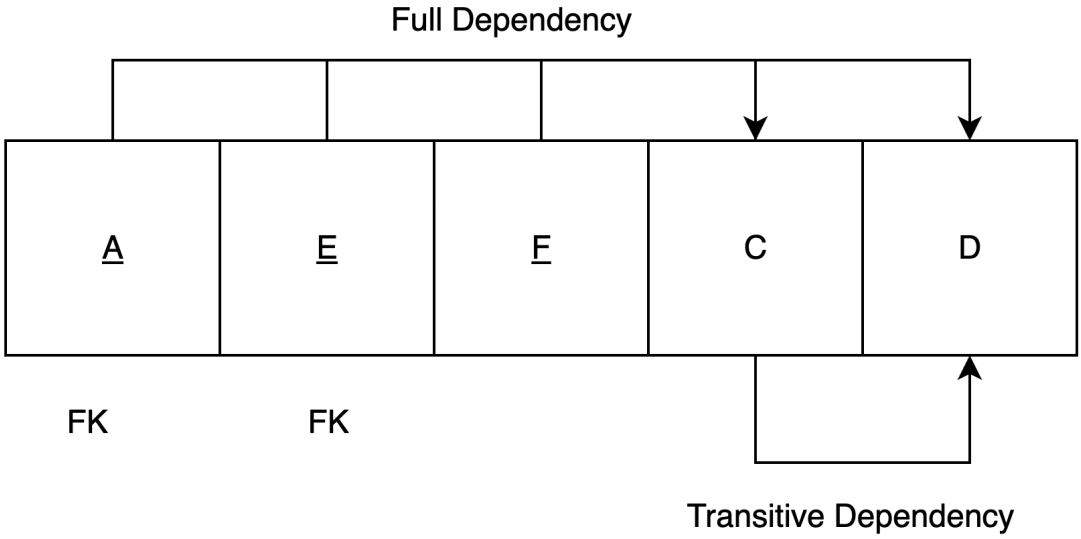
A)

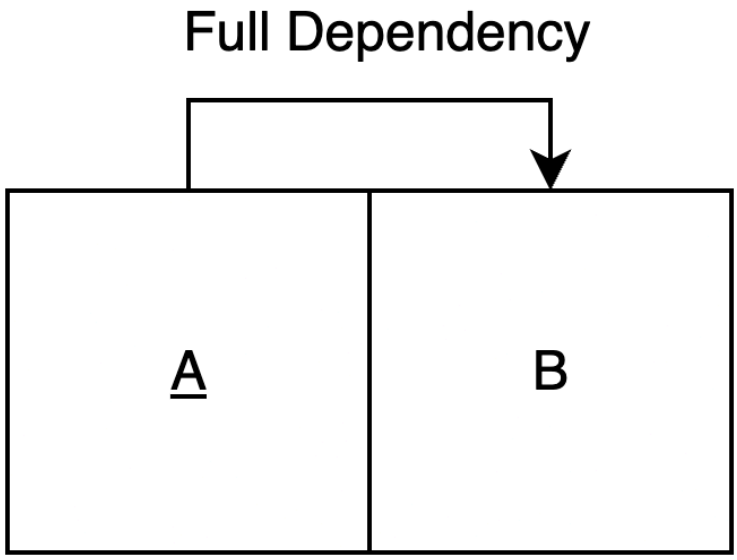
Primary Key (A, E, F)



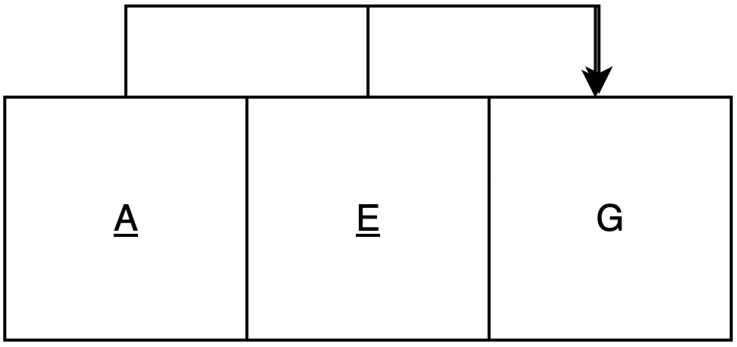
b)

2NF


Full Dependency
A
E
F
C
D
FK
FK
Transitive Dependency

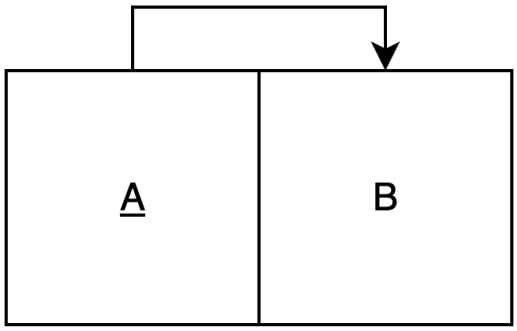

Full Dependency
A
B

Full Dependency

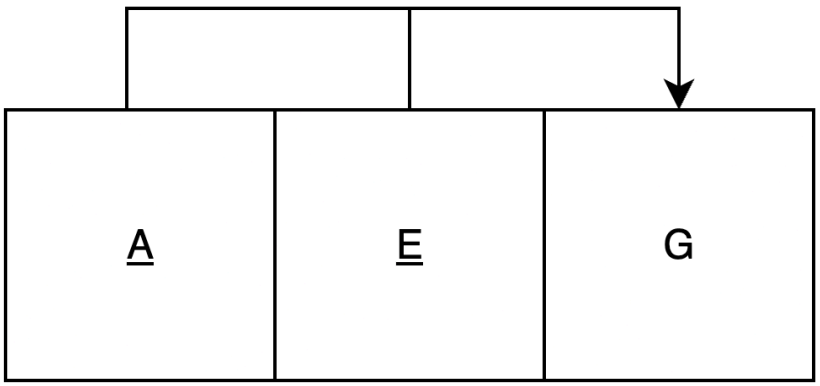


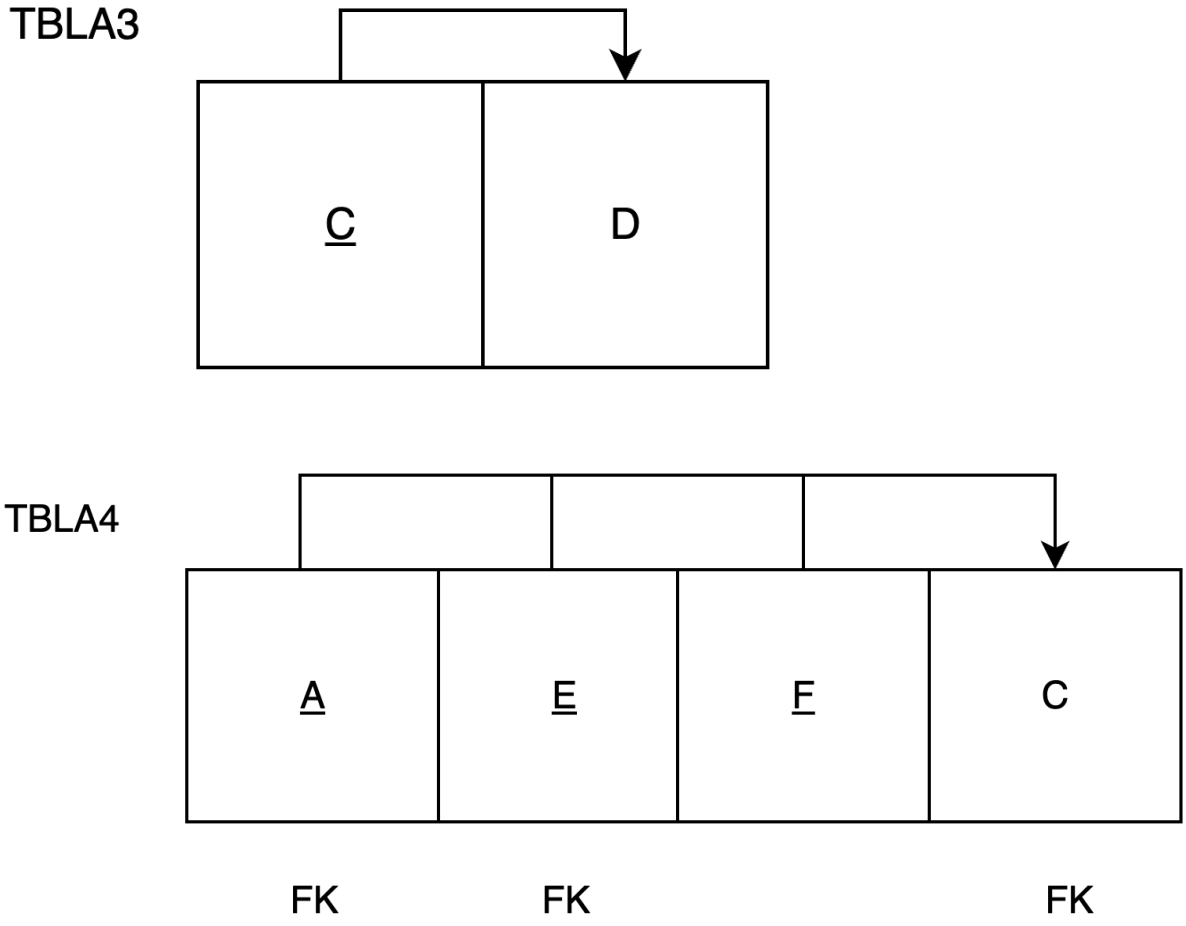
3NF

TBLA1



TBLA2





c)

```
CREATE TABLE TBLA1 (
  A INTEGER PRIMARY KEY,
  B TEXT
);

CREATE TABLE TBLA2 (
  A INTEGER,
  E INTEGER,
  G TEXT,
  PRIMARY KEY (A, E)
);

CREATE TABLE TBLA3 (
```

```
  C INTEGER PRIMARY KEY,
  D TEXT
);

CREATE TABLE TBLA4 (
  A INTEGER,
  E INTEGER,
  F INTEGER,
  C INTEGER,
  PRIMARY KEY (A, E, F),
  FOREIGN KEY (A)     REFERENCES TBLA1(A),
  FOREIGN KEY (A, E)  REFERENCES TBLA2(A, E),
  FOREIGN KEY (C)     REFERENCES TBLA3(C)
);
```

D)

If E is no longer needed in A,E to G (so it becomes A to G)
Then table TBLA2(A,E,G) is unnecessary. Combine G into the A-keyed table:
Combine: TBLA1(A,B) and TBLA2(A,E,G) to TBLA1(A,B,G)` (PK A).
Drop the (A,E) foreign key from TBLA4; keep A and C FKs.

Final 3NF (with A to G)
TBLA1(A, B, G) - PK A
TBLA3(C, D) - PK C
TBLA4(A, E, F, C) - PK (A, E, F); FKs: A to TBLA1, C to TBLA3
Create order: TBLA1, TBLA2, TBLA3

```
/*1*/
CREATE TABLE TBLA1 (
  A INTEGER PRIMARY KEY,
  B TEXT,
  G TEXT
);

/*2*/
CREATE TABLE TBLA2 (
  C INTEGER PRIMARY KEY,
  D TEXT
);

/*3*/
CREATE TABLE TBLA3 (
```

```
  A INTEGER,
  E INTEGER,
  F INTEGER,
  C INTEGER,
  PRIMARY KEY (A, E, F),
  FOREIGN KEY (A) REFERENCES TBLA1(A),
  FOREIGN KEY (C) REFERENCES TBLA2(C)
);
```

E)

The “Employee–Department” example from Lecture 10 guided my change. In that example, EmpNo fully determines all attributes, but Dept determines DeptName and Location. The solution removes the transitive part into a separate Department table and leaves a foreign key in Employee. This shows how to redraw dependencies, split tables, and place foreign keys when a determinant changes.
Lecture 9 set the rules I followed.
Draw full dependencies at the top.
In 3NF there are no transitive dependencies. When a determinant changes, create a table with that determinant as a key, then move its dependents, and recheck foreign keys.
Applying those steps to my diagram: once A, E→G becomes A→G, E is no longer part of the determinant for G. The lecture rule says the table structure must reflect the new determinant. I merged G into the A-keyed table and removed the composite foreign key (A, E) from the bridge, keeping only the foreign keys that still reference valid keys. That mirrors the way the EmpNo–Dept example moves DeptName and Location under Dept and keeps a single Dept foreign key in Employee.
Why this helped: the lecture example gave a concrete pattern for altering tables when determinants change, and Lecture 9 clarified the check list for 3NF diagrams and arrows. I used the same pattern to justify combining TBLA1 and TBLA2 into TBLA1(A, B, G), and to update foreign keys in TBLA4, while keeping TBLA3(C, D) unchanged.

F)

E→F meansF depends on E only. Keys must be minimal. So F is not part of a minimal key with E.
Changes to the 3NF tables
Keep TBLA1(A, B). PK A.
Keep TBLA2(A, E, G). PK A, E.

Keep TBLA3(C, D). PK C.
Add EF(E, F). PK E.
Update TBLA4 to TBLA4(A, E, C). PK A, E. FKs, A to TBLA1. C to TBLA3. Optionally add FK E to EF.
Why this is correct
E determines F, so A, E, F is not minimal. The key becomes A, E.
3NF requires each determinant to be the key of some table. EF satisfies this.
After the change every non-key depends on the key, the whole key, and nothing but the key.
No transitive dependencies remain. Draw all final dependencies on the top rail.

—----------

2

A)

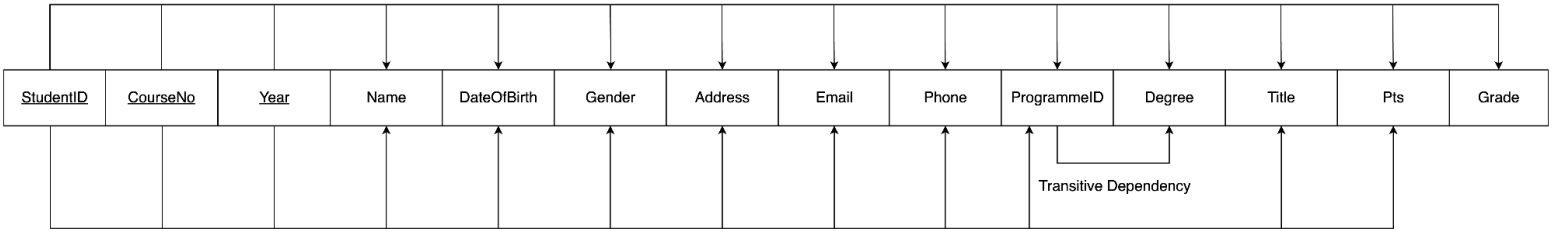
PK: (StudentID, CourseNo, Year)

B)

1NF

PK: (StudentID, CourseNo, Year)

Full Dependency



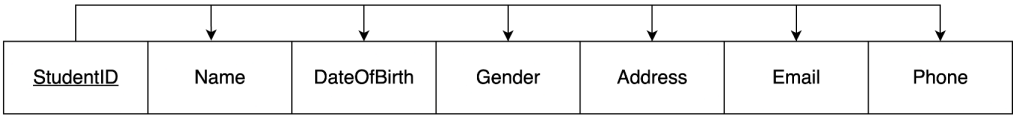
Partial Dependency

C)

2NF

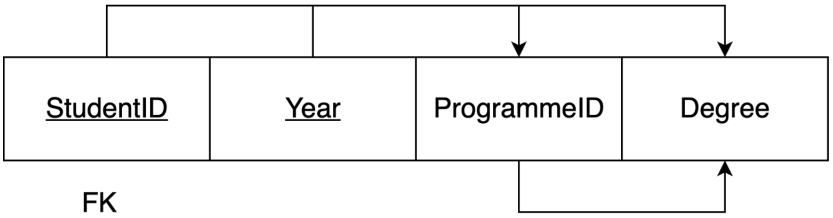
2nf

Table Name: STUDENT



FK

Table Name: STUDENT_PROGRAMME_YEAR



FK

Transitive Dependency

Table Name: COURSE_OFFERING

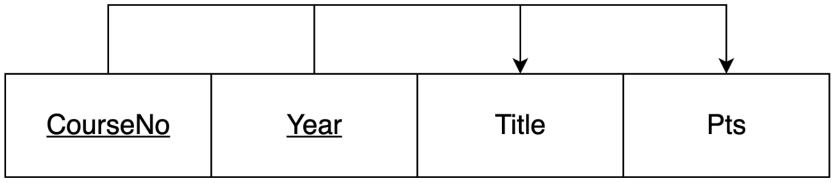
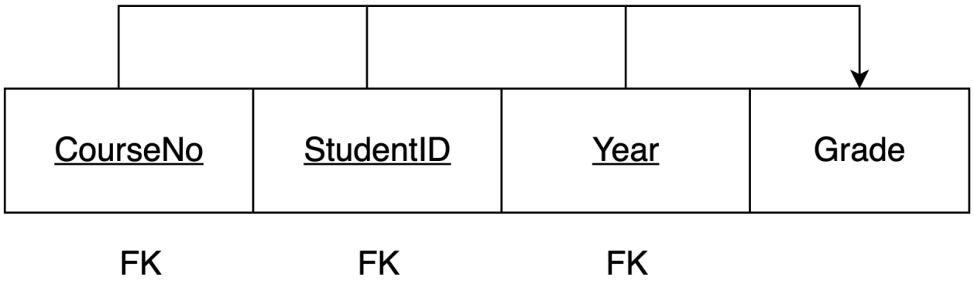
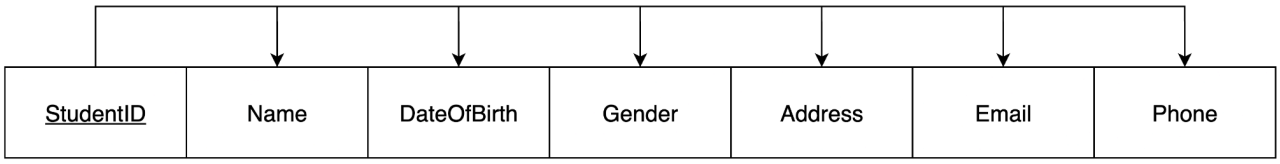


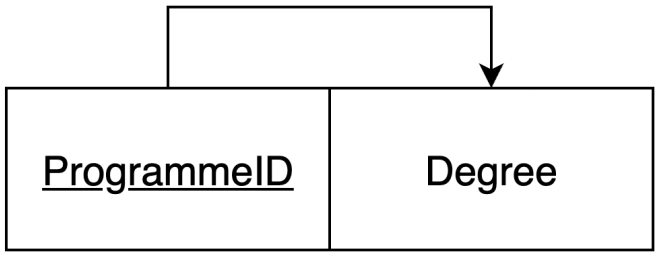
Table Name: ENROLMENT



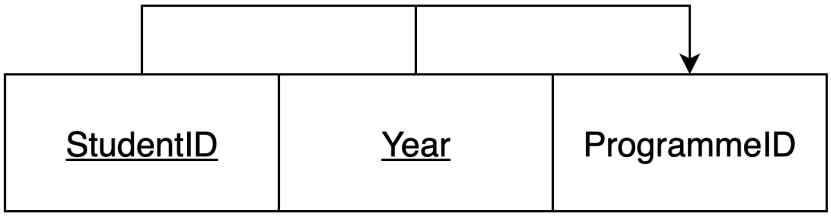
3NF
Table 1: STUDENT
Primary key: StudentID
Foreign key: none



—
Table 2: PROGRAMME
Primary key: ProgrammeID
Foreign key: none



—
Table 3: STUDENT_PROGRAMME_YEAR
Primary key: StudentID + Year
Foreign key:
StudentID (to STUDENT)
ProgrammeID (to PROGRAMME)

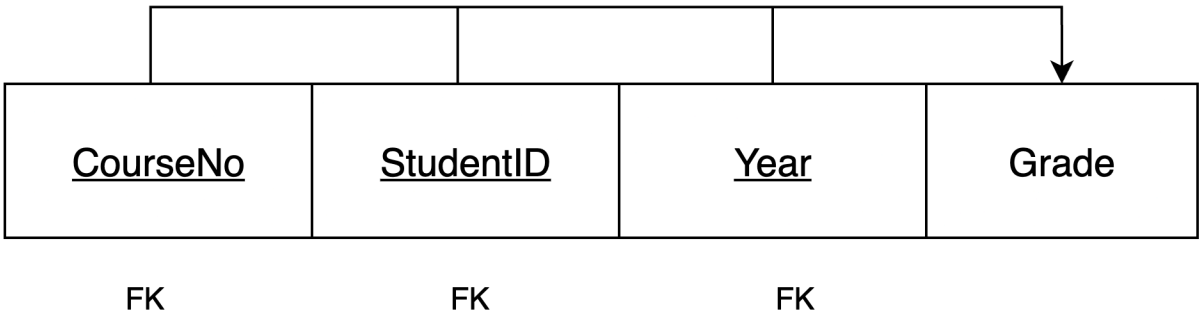


—

Table 4: COURSE_OFFERING
Primary key: CourseNo + Year
Foreign key: none

—

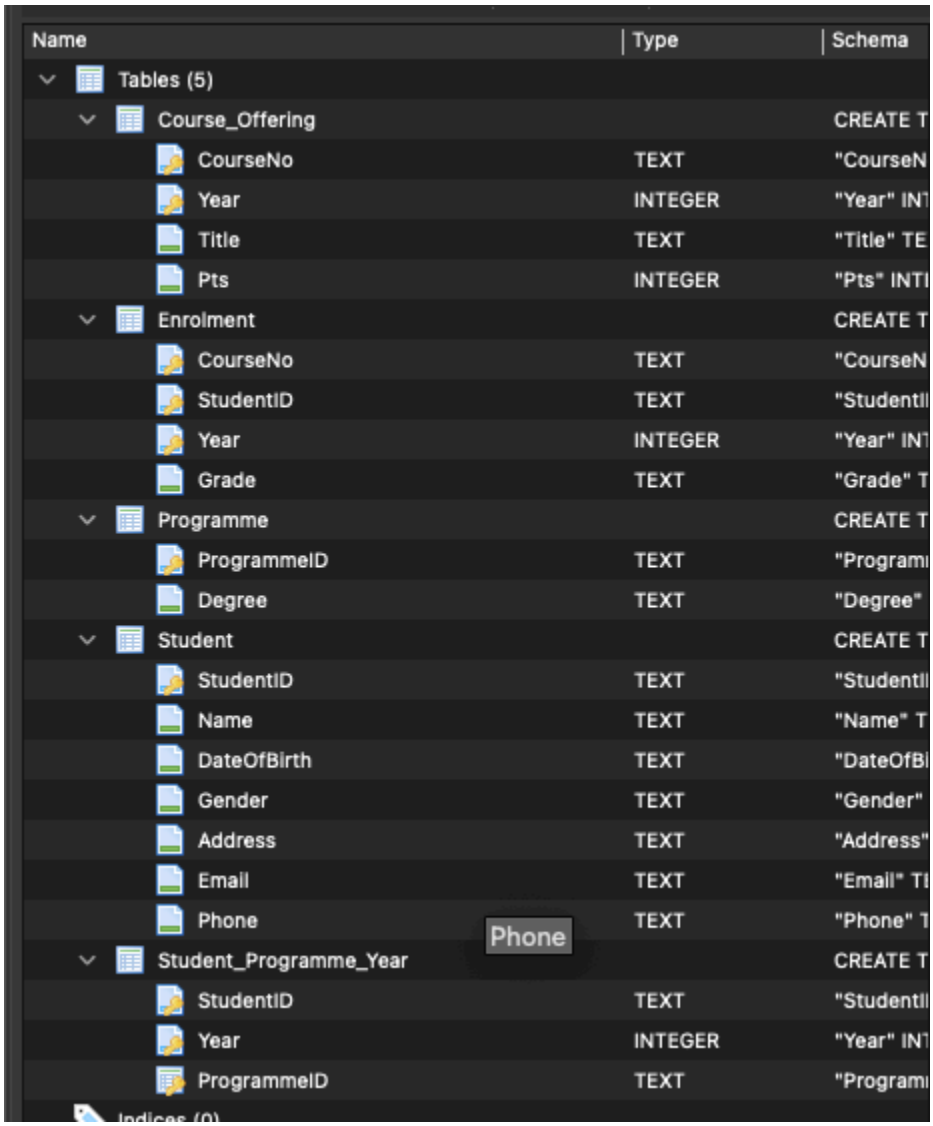
Table 5: ENROLMENT
Primary key: StudentID + CourseNo + Year
Foreign key:
StudentID (to STUDENT)
CourseNo + Year (to COURSE_OFFERING)
StudentID + Year (to STUDENT_PROGRAMME_YEAR)



d)

```
/*1*/
CREATE TABLE Student (
  StudentID   TEXT PRIMARY KEY,
  Name        TEXT,
  DateOfBirth TEXT,
  Gender      TEXT,
  Address     TEXT,
  Email       TEXT,
  Phone       TEXT
);

/*2*/
CREATE TABLE Programme (
  ProgrammeID TEXT PRIMARY KEY,
  Degree      TEXT
);

/*3*/
CREATE TABLE Course_Offering (
  CourseNo TEXT,
  Year     INTEGER,
  Title    TEXT,
  Pts      INTEGER,
  PRIMARY KEY (CourseNo, Year)
);

/*4*/
CREATE TABLE Student_Programme_Year (
  StudentID   TEXT,
  Year        INTEGER,
  ProgrammeID TEXT,
  PRIMARY KEY (StudentID, Year),
  FOREIGN KEY (StudentID)   REFERENCES Student(StudentID),
  FOREIGN KEY (ProgrammeID) REFERENCES Programme(ProgrammeID)
);

/*5*/
CREATE TABLE Enrolment (
  CourseNo  TEXT,
  StudentID TEXT,
```

```
  Year      INTEGER,
  Grade     TEXT,
  PRIMARY KEY (StudentID, CourseNo, Year),
  FOREIGN KEY (CourseNo, Year)
    REFERENCES Course_Offering(CourseNo, Year),
  FOREIGN KEY (StudentID, Year)
    REFERENCES Student_Programme_Year(StudentID, Year),
  FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID)
);
```



e)

```
/* Student */
```

```
INSERT INTO Student (StudentID, Name, DateOfBirth, Gender, Address, Email, Phone)
VALUES
('007007','James Bond','07/07/1977','Male',
 '10 Downing Street Wellington','JB007@gmail.com','+64 21 123 4567');

/*2) Programmes*/
INSERT INTO Programme (ProgrammeID, Degree) VALUES
('PROG002','Bachelor of Technology'),
('PROG001','Bachelor of Commerce');

/*3) Course offerings (by year; preserves historic titles and points)*/
INSERT INTO Course_Offering (CourseNo, Year, Title, Pts) VALUES
('INF0151',2019,'Databases',15),
('ECOM130',2019,'Microeconomic Principles',15),
('FCOM111',2019,'Government Law and Business',15),
('INF0101',2019,'Foundations of Info Systems',15),
('INF0141',2018,'Systems Analysis',15),
('INF0151',2018,'Databases & SQL',15),
('MGMT101',2018,'Introduction to Management',15),
('QUAN102',2018,'Statistics for Business',15);

/*4) Student's programme per year*/
INSERT INTO Student_Programme_Year (StudentID, Year, ProgrammeID) VALUES
('007007',2019,'PROG002'),
('007007',2018,'PROG001');

/*5) Enrolments with grades*/
INSERT INTO Enrolment (CourseNo, StudentID, Year, Grade) VALUES
('INF0151','007007',2019,'A+'),
('ECOM130','007007',2019,'A+'),
('FCOM111','007007',2019,'B+'),
('INF0101','007007',2019,'A+'),
('INF0141','007007',2018,'A-'),
('INF0151','007007',2018,'WD'),
('MGMT101','007007',2018,'B+'),
('QUAN102','007007',2018,'A+');


/*  Student */
SELECT * FROM Student;
```

StudentID	Name	DateOfBirth	Gender	Address	Email	Phone

007007	James Bond	07/07/1977	Male	10 Downing Street Wellington	JB007@gmail.com	+64 21 123 4567

/* Programme */
SELECT * FROM Programme;

ProgrammeID	Degree
PROG002	Bachelor of Technology
PROG001	Bachelor of Commerce

/* Course_Offering */
SELECT * FROM Course_Offering ORDER BY Year, CourseNo;

CourseNo	Year	Title	Pts
INF0141	2018	Systems Analysis	15
INF0151	2018	Databases & SQL	15
MGMT101	2018	Introduction to Management	15
QUAN102	2018	Statistics for Business	15
ECOM130	2019	Microeconomic Principles	15
FCOM111	2019	Government Law and Business	15
INF0101	2019	Foundations of Info Systems	15
INF0151	2019	Databases	15

/* Student_Programme_Year */
SELECT * FROM Student_Programme_Year ORDER BY Year;

StudentID	Year	ProgrammeID
007007	2018	PROG001

007007	2019	PROG002

/* Enrolment */
SELECT * FROM Enrolment ORDER BY Year, CourseNo;

CourseNo	StudentID	Year	Grade
INF0141	007007	2018	A-
INF0151	007007	2018	WD
MGMT101	007007	2018	B+
QUAN102	007007	2018	A+
ECOM130	007007	2019	A+
FCOM111	007007	2019	B+
INF0101	007007	2019	A+
INF0151	007007	2019	A+

F)

```
/*Transaction log table */
CREATE TABLE TxnLog (
  log_id INTEGER PRIMARY KEY AUTOINCREMENT,
  action TEXT,
  student_id TEXT,
  course_no TEXT,
  year INTEGER,
  old_grade TEXT,
  new_grade TEXT,
  ts TEXT DEFAULT current_timestamp
);
```

```
/* TCL to update grade for student 007007 in INF0141 from A- to B with logging */
BEGIN TRANSACTION;
```

```
/* BEGIN */
INSERT INTO TxnLog(action, student_id, course_no, year)
VALUES ('BEGIN', '007007', 'INF0141', 2018);

/* BEFORE snapshot */
INSERT INTO TxnLog(action, student_id, course_no, year, old_grade)
SELECT 'BEFORE', '007007', 'INF0141', 2018, Grade
FROM Enrolment
WHERE StudentID = '007007'
  AND CourseNo  = 'INF0141'
  AND Year      = 2018;

/* Guarded UPDATE */
UPDATE Enrolment
SET Grade = 'B'
WHERE StudentID = '007007'
  AND CourseNo  = 'INF0141'
  AND Year      = 2018
  AND Grade     = 'A-';

/* Verify outcome for marking */
SELECT COUNT(*) AS rows_changed
FROM Enrolment
WHERE StudentID = '007007'
  AND CourseNo  = 'INF0141'
  AND Year      = 2018
  AND Grade     = 'B';

/* AFTER and COMMIT block */
INSERT INTO TxnLog(action, student_id, course_no, year, old_grade, new_grade)
SELECT 'AFTER', '007007', 'INF0141', 2018, 'A-', 'B';

INSERT INTO TxnLog(action, student_id, course_no, year)
VALUES ('COMMIT', '007007', 'INF0141', 2018);

COMMIT;

/*Enrolment check*/
SELECT StudentID, CourseNo, Year, Grade
FROM Enrolment
WHERE StudentID='007007' AND CourseNo='INF0141' AND Year=2018;
```

StudentID	CourseNo	Year	Grade

007007	INF0141	2018	B

```
/*transaction log*/
SELECT action, old_grade, new_grade, ts
FROM TxnLog
WHERE student_id='007007' AND course_no='INF0141';
```

action	old_grade	new_grade	ts
BEGIN			2025-10-12 08:05:41
BEFORE	A-		2025-10-12 08:05:41
AFTER	A-	B	2025-10-12 08:05:41
COMMIT			2025-10-12 08:05:41

ACID tie-back
Atomicity: one BEGIN TRANSACTION to COMMIT block, or ROLLBACK.
Consistency: guarded UPDATE with Grade = 'A-'.
Isolation: changes visible only after COMMIT.
Durability: committed grade and TxnLog rows persist via current_timestamp.

g)

```
SELECT
  s.StudentID,
  s.Name,
  ROUND(
    SUM(co.Pts *
        CASE e.Grade
          WHEN 'A+' THEN 4.0
          WHEN 'A'  THEN 4.0
          WHEN 'A-' THEN 3.7
          WHEN 'B+' THEN 3.3
          WHEN 'B'  THEN 3.0
          WHEN 'B-' THEN 2.7
          WHEN 'C+' THEN 2.3
          WHEN 'C'  THEN 2.0
```

```
          WHEN 'D'  THEN 1.0
          WHEN 'F'  THEN 0.0
          ELSE NULL
        END
    ) / SUM(co.Pts)
  , 2) AS OverallGPA
FROM Student AS s
JOIN Enrolment AS e
  ON e.StudentID = s.StudentID
JOIN Course_Offering AS co
  ON co.CourseNo = e.CourseNo
 AND co.Year    = e.Year
WHERE s.StudentID = '007007'
  AND e.Grade <> 'WD'
GROUP BY s.StudentID, s.Name;
```

StudentID	Name	OverallGPA
007007	James Bond	3.66

3.

A)

Update anomaly

A change to one fact needs many edits because the fact is copied in many rows. Miss one edit, and your data becomes inconsistent.

How the flat table causes it

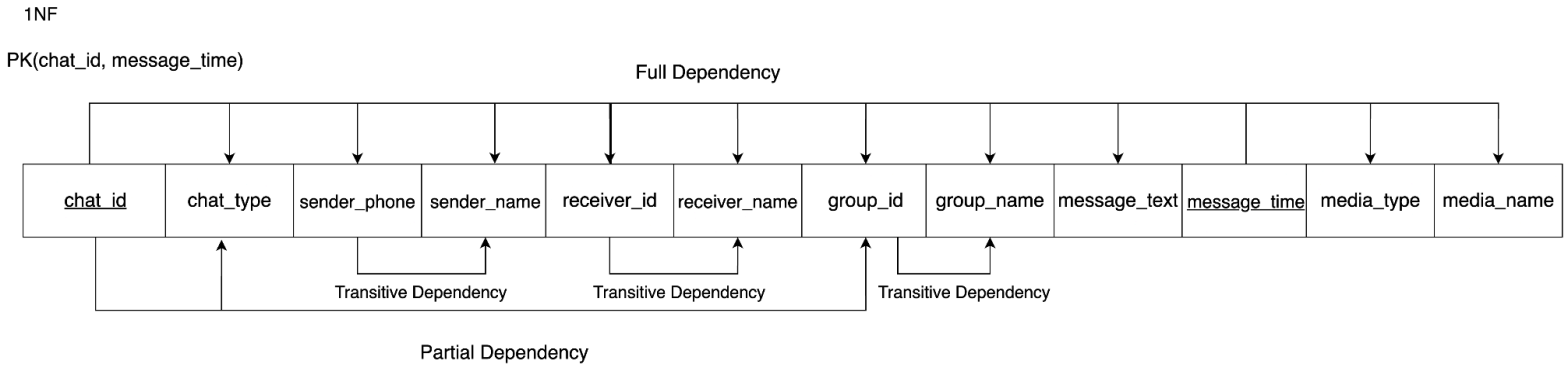
User facts repeat. The same person's name appears in every row where that phone number shows up, as sender_name and as receiver_name. Example, C101 has Alice and Bob twice. If Bob changes his name to Robert, you must update every row with +1987654321. If one row is missed, the table shows both "Bob" and "Robert" for the same phone.

• Group facts repeat. The same group name appears in every message for that group, often in two columns, receiver_name and group_name. Example, G202 "Group1" appears on three C202 rows. A rename to "Project Team" needs multiple edits. If one row is missed, some messages still show "Group1."

The same group key is stored twice. Group chats copy G202 in both receiver_id and group_id. A change needs two edits per row, which invites mistakes.
Fix
Store each fact once. Put users in User(phone, name). Put groups in Group(group_id, group_name). Keep messages in a Message table that references phones and group_ids. Now one edit changes one row, and you avoid the update anomaly.

B)

PK(chat_id, message_time)



C)

2NF

Table 1: Chat

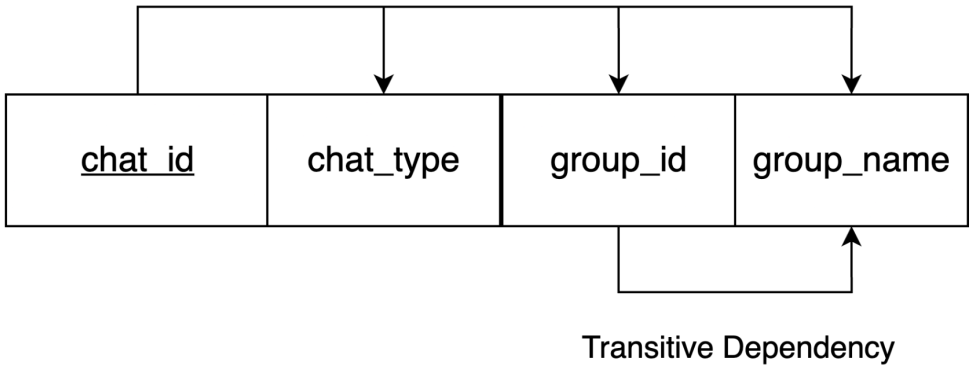
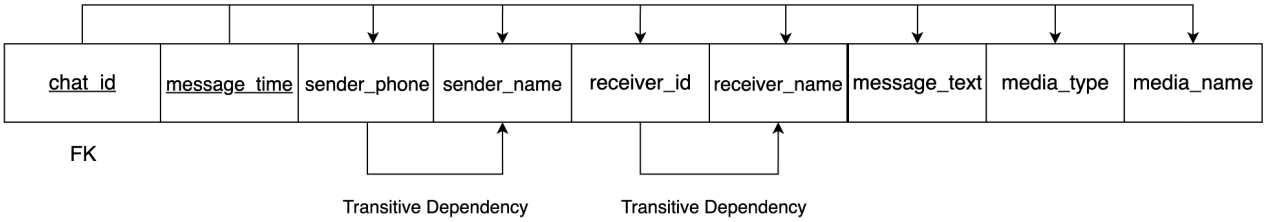


Table 2: MESSAGE



–

3nf

Table 1: GROUP

Primary key: group_id

Foreign key: none

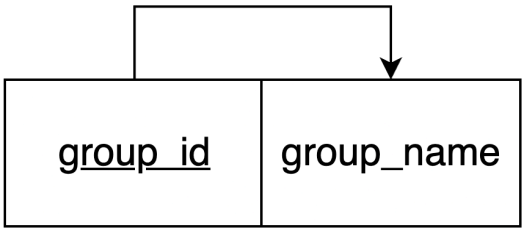


Table 2: CHAT

Primary key: chat_id

Foreign key: group_id (to GROUP)

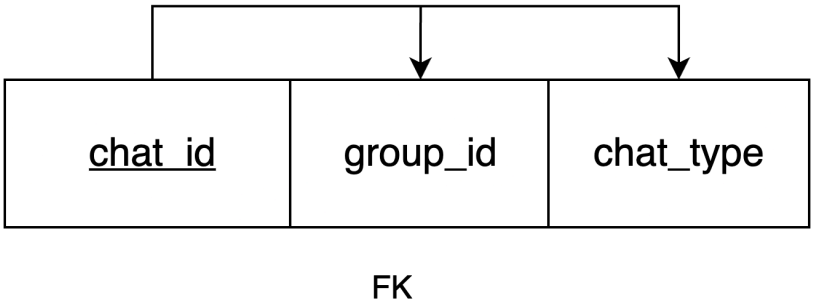


Table 3: USER

Primary key: sender_phone

Foreign key: None

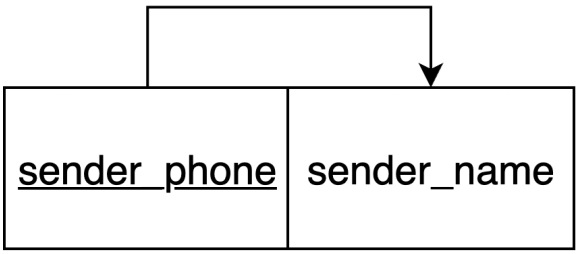


Table 4: RECEIVER
Primary key: receiver_id
Foreign key: None

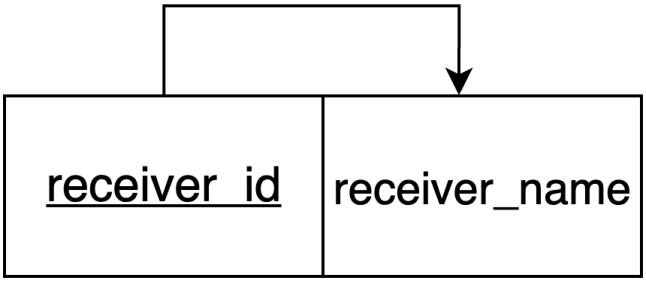
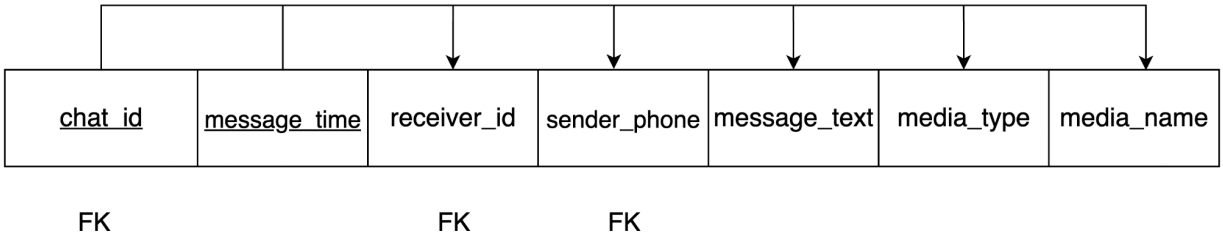
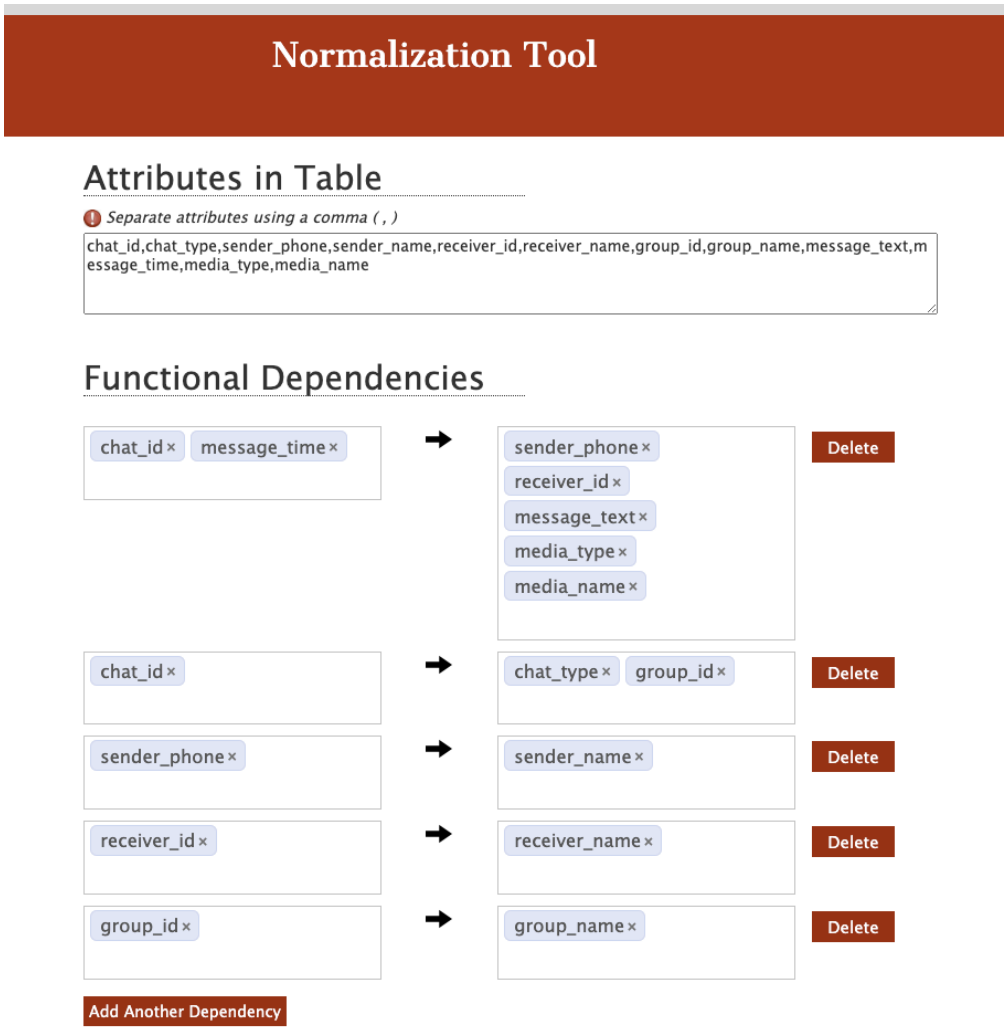


Table 5: MESSAGE
Primary key: chat_id + message_time
Foreign key: chat_id (to CHAT)
receiver_id (to RECEIVER)
sender_phone (to USER)



D)


Normalization Tool
Attributes in Table
Separate attributes using a comma (,)
chat_id,chat_type,sender_phone,sender_name,receiver_id,receiver_name,group_id,group_name,message_text,message_time,media_type,media_name
Functional Dependencies
chat_id × message_time ×
sender_phone × receiver_id × message_text × media_type × media_name ×
Delete
chat_id ×
chat_type × group_id ×
Delete
sender_phone ×
sender_name ×
Delete
receiver_id ×
receiver_name ×
Delete
group_id ×
group_name ×
Delete
Add Another Dependency

E)

1NF to 3NF

Attributes

group_id group_name

Functional Dependencies

group_id → group_name

Attributes

chat_id chat_type group_id

Functional Dependencies

chat_id → group_id chat_type

Attributes

sender_phone sender_name

Functional Dependencies

sender_phone → sender_name

Attributes

receiver_id receiver_name

Functional Dependencies

receiver_id → receiver_name

Attributes

chat_id sender_phone receiver_id message_text message_time media_type media_name

Functional Dependencies

chat_id message_time → receiver_id sender_phone message_text media_type media_name

Table 1: GROUP
Primary key: group_id
Foreign key: none

Table 2: CHAT
Primary key: chat_id
Foreign key: group_id (to GROUP)

Table 3: USER
Primary key: sender_phone
Foreign key: None

Table 4: RECEIVER
Primary key: receiver_id
Foreign key: None

Table 5: MESSAGE
Primary key: chat_id + message_time
Foreign key: chat_id (to CHAT)
receiver_id (to RECEIVER)
sender_phone (to USER)

F)

```
/* 1) GROUP */
CREATE TABLE "Group" (
  group_id   TEXT PRIMARY KEY,
  group_name TEXT
);

/* 2) CHAT */
CREATE TABLE Chat (
  chat_id   TEXT PRIMARY KEY,
  group_id  TEXT,
  chat_type TEXT,
  FOREIGN KEY (group_id) REFERENCES "Group"(group_id)
);

/* 3) USER */
CREATE TABLE "User" (
  sender_phone TEXT PRIMARY KEY,
  sender_name  TEXT
);
```

```
/* 4) RECEIVER */
CREATE TABLE Receiver (
  receiver_id   TEXT PRIMARY KEY,
  receiver_name TEXT
);

/* 5) MESSAGE */
CREATE TABLE Message (
  chat_id       TEXT,
  message_time  TEXT,
  receiver_id   TEXT,
  sender_phone  TEXT,
  message_text  TEXT,
  media_type    TEXT,
  media_name    TEXT,
  PRIMARY KEY (chat_id, message_time),
  FOREIGN KEY (chat_id)      REFERENCES Chat(chat_id),
  FOREIGN KEY (receiver_id)  REFERENCES Receiver(receiver_id),
  FOREIGN KEY (sender_phone) REFERENCES "User"(sender_phone)
);
```

G)

```
BEGIN TRANSACTION;

/* GROUP */
INSERT INTO "Group"(group_id, group_name) VALUES ('G202','Group1');
INSERT INTO "Group"(group_id, group_name) VALUES ('G303','StudyGroup');

/* CHAT */
INSERT INTO Chat(chat_id, group_id, chat_type) VALUES ('C101', NULL, 'individual');
INSERT INTO Chat(chat_id, group_id, chat_type) VALUES ('C202', 'G202','group');
INSERT INTO Chat(chat_id, group_id, chat_type) VALUES ('C303', 'G303','group');

/* USER */
INSERT INTO "User"(sender_phone, sender_name) VALUES ('+1234567890','Alice');
INSERT INTO "User"(sender_phone, sender_name) VALUES ('+1987654321','Bob');
INSERT INTO "User"(sender_phone, sender_name) VALUES ('+1111111111','Charlie');
INSERT INTO "User"(sender_phone, sender_name) VALUES ('+3333333333','David');
INSERT INTO "User"(sender_phone, sender_name) VALUES ('+4444444444','Emma');
INSERT INTO "User"(sender_phone, sender_name) VALUES ('+5555555555','Frank');

/* RECEIVER */
```

```
INSERT INTO Receiver(receiver_id, receiver_name) VALUES ('+1987654321','Bob');
INSERT INTO Receiver(receiver_id, receiver_name) VALUES ('+1234567890','Alice');
INSERT INTO Receiver(receiver_id, receiver_name) VALUES ('G202','Group1');
INSERT INTO Receiver(receiver_id, receiver_name) VALUES ('G303','StudyGroup');

/* MESSAGE */
INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C101','2023-10-21 09:15:05','+1987654321','+1234567890','Hey, how are you?',
NULL, NULL);

INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C101','2023-10-21 09:17:10','+1234567890','+1987654321','I''m fine, thanks', NULL,
NULL);

INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C202','2023-10-21 10:00:15','G202','+1111111111','Meeting at 5 pm', NULL, NULL);

INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C202','2023-10-21 10:05:20','G202','+3333333333','Sending file','image','notes.png');

INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C202','2023-10-21 10:10:30','G202','+4444444444','Sure, see you', NULL, NULL);

INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C303','2023-10-22 14:00:45','G303','+1234567890','Let''s start', NULL, NULL);

INSERT INTO Message(chat_id, message_time, receiver_id, sender_phone, message_text,
media_type, media_name)
VALUES ('C303','2023-10-22 14:05:50','G303','+5555555555','Check this
out','video','lecture.mp4');

COMMIT;
```

—-

Tables

Group:

group_id	group_name
G202	Group1
G303	StudyGroup

Chat:

chat_id	group_id	chat_type
C101		individual
C202	G202	group
C303	G303	group

User:

sender_phone	sender_name
+1234567890	Alice
+1987654321	Bob
+1111111111	Charlie
+3333333333	David
+4444444444	Emma
+5555555555	Frank

Receiver:

receiver_id	receiver_name
+1987654321	Bob
+1234567890	Alice
G202	Group1

G303	StudyGroup

Message:

chat_id	message_time	receiver_id	sender_phone	message_text	media_type	media_name
C101	2023-10-21 09:15:05	+1987654321	+1234567890	Hey, how are you?		
C101	2023-10-21 09:17:10	+1234567890	+1987654321	I'm fine, thanks		
C202	2023-10-21 10:00:15	G202	+1111111111	Meeting at 5 pm		
C202	2023-10-21 10:05:20	G202	+3333333333	Sending file	image	notes.png
C202	2023-10-21 10:10:30	G202	+4444444444	Sure, see you		
C303	2023-10-22 14:00:45	G303	+1234567890	Let's start		
C303	2023-10-22 14:05:50	G303	+5555555555	Check this out	video	lecture.mp4

H)

```
CREATE VIEW v_studygroup_media AS
SELECT
c.chat_id,
m.message_time,
u.sender_name,
m.message_text,
m.media_type,
m.media_name,
g.group_name
FROM Message AS m
JOIN Chat AS c ON c.chat_id = m.chat_id
JOIN "Group" AS g ON g.group_id = c.group_id
JOIN "User" AS u ON u.sender_phone = m.sender_phone
```

```
WHERE g.group_name = 'StudyGroup'
AND m.media_type IS NOT NULL
ORDER BY m.message_time;
```

chat_id	message_time	sender_name	message_text	media_type	media_name	group_name
C303	2023-10-22 14:05:50	Frank	Check this out	video	lecture.mp4	StudyGroup

l)

```
CREATE TABLE Group_Member (
  group_id     TEXT,
  sender_phone TEXT,
  join_date    TEXT DEFAULT current_timestamp,
  PRIMARY KEY (group_id, sender_phone),
  FOREIGN KEY (group_id)     REFERENCES "Group"(group_id),
  FOREIGN KEY (sender_phone) REFERENCES "User"(sender_phone)
);
```